UNIVERSIDAD DE LA EMPRESA

Bases de Datos III

Trabajo Obligatorio
Curso 2026
Licenciatura e Ingeniería en Informática
Total: 100 puntos - Mínimo de aprobación: 50 puntos

Indicaciones Generales

- Los grupos deberán tener cinco (5) integrantes. En caso de que la cantidad total de estudiantes **no** sea múltiplo de 5, se admitirán **excepcionalmente** grupos de otra cantidad de integrantes.
- La fecha de entrega es el ***Lunes 16 de noviembre de 2026*** hasta las 23:59 hs. Se debe entregar el proyecto en eclipse (vía e-mail a la casilla del docente) conteniendo:
  1. El programa `main` encargado de crear la base de datos para el sistema de registro de notas de alumnos, según las indicaciones del ejercicio 2 del práctico 4.
  2. La estructura de packages definida en el ejercicio 2 del práctico 5, con la implementación de todas las clases e interfaces implementadas entre los ejercicios 2 y 4 del práctico 5.
  3. Los packages adicionales necesarios para contener la implementación de las nuevas clases e interfaces adicionales para un ***segundo mecanismo de persistencia*** que se solicita en este trabajo obligatorio (ver más adelante, puntos 5 al 9 de la letra).
- En la clase siguiente a la entrega (***Martes 17 de noviembre*** en Montevideo y Punta del Este, ***Viernes 20 de noviembre*** en Colonia), cada grupo realizará una ***defensa oral*** del trabajo, que consistirá en una demostración de la ejecución de la aplicación construida junto con una serie de preguntas relativas tanto al diseño de la solución como a la ejecución de la aplicación. **No** se requiere realizar presentación alguna, solamente la demo (ejecutando todas las funcionalidades sobre ambos mecanismos de persistencia) y las preguntas. La defensa deberá ser realizada en forma obligatoria por **todos** los integrantes del grupo. El desempeño de cada integrante en la defensa influirá en la nota final del trabajo.

Características del Trabajo

El trabajo obligatorio consiste en la implementación **completa** de la aplicación resultante de la realización de los siguientes ejercicios de los prácticos del curso (más otros puntos adicionales que se solicitan más adelante):

1. Ejercicio 2 del Práctico 4 (creación de la base de datos para el sistema de registro de notas).
2. Ejercicio 4 del Práctico 4 (creación del Pool de Conexiones para manejo de concurrencia).
3. Ejercicio 2 del Práctico 5 (implementación de la aplicación en 3 capas persistiendo en la base de datos creada en el punto 1).
4. Ejercicio 4 del Práctico 5 (incorporación de manejo de concurrencia a la aplicación en 3 capas).

Se deben resolver los ejercicios indicados en forma ***incremental***, siguiendo el orden en que fueron propuestos. El resultado deberá ser una ***aplicación en 3 capas*** con las siguientes características:

- Debe poseer la ***arquitectura física 3-Tier*** descrita en el Ejercicio 2 del Práctico 4.
- Debe realizar manejo de concurrencia utilizando ***Pool de Conexiones***.
- Debe utilizar los patrones ***Facade***, ***Value Object***, ***MVC***, ***DAO*** vistos en el curso.

Una vez completados los ejercicios indicados, se deberá incorporar a la aplicación el uso del patrón ***Abstract Factory***, de modo que también se pueda persistir toda la información en un ***segundo*** mecanismo de persistencia, que consistirá en una estructura de ***archivos de texto plano*** en disco. Dichos archivos deberán poder visualizarse (por fuera de la aplicación) mediante algún editor adecuado (ejemplo: `notepad`). Habrá un archivo llamado `alumnos.txt` conteniendo los datos de todos los alumnos (cédula, nombre y apellido de c/u) ordenados por cédula. Dentro del archivo, los datos de cada alumno estarán en una línea aparte, separados por el carácter `|`. Por ejemplo:

Este material es de uso exclusivo para los cursos impartidos por Universidad de la Empresa

```
12345678|Matilda|Wormwood
25554449|Charlie|Brown
39876542|Dora|Márquez
51112223|Daniel|Mitchell
66666666|Merlina|Addams
```

A su vez, por alumno habrá otro archivo conteniendo sus registros de notas (número, actividad y nota de c/u) ordenados por número. El nombre del archivo será la cédula antecedida del prefijo `registros`, y tendrá extensión `.txt`. Dentro del archivo, los datos de cada registro estarán en una línea aparte, separados por el carácter `|`. Por ejemplo: el archivo de Matilda Woormwood se llamará `registros12345678.txt` y (suponiendo que realizó tres actividades) su contenido lucirá así:

```
1|Matematica|12
2|Lengua|9
3|Dibujo|10
```

La aplicación deberá ser capaz de persistir ***indistintamente*** en uno u otro medio de persistencia (la BD del práctico 4 o la estructura de archivos de texto plano). En el código fuente de la fachada **no** deberá haber indicación explícita del medio de persistencia usado. El cambio de un medio a otro deberá manejarse con ***archivos de configuración***, del modo visto en el curso. Se deberá agregar a la aplicación desarrollada hasta el momento la implementación que se pide en los siguientes puntos:

5. Implementación de las clases `ConexionTxt` y `PoolConexionesTxt`, las cuales deberán implementar las mismas interfaces `IConexion` e `IPoolConexiones` vistas en el curso y corresponden a un ***segundo*** pool de conexiones, que tendrá una implementación ***diferente*** a la desarrollada para la base de datos. Servirá para controlar el manejo de concurrencia sobre la estructura de archivos correspondiente al segundo mecanismo de persistencia.

- Los métodos `obtenerConexion` y `liberarConexion` tendrán el comportamiento usual de un ***monitor de lectura y escritura*** similar al utilizado en Taller II. Si la conexión se pide para un requerimiento que **modifica** los archivos, se considerará como un ***escritor***. En cambio, si es un requerimiento que **no** modifica archivos, se considerará como un ***lector***.
- La clase `PoolConexionesTxt` deberá definir los atributos necesarios para el manejo usual de un monitor de lectura y escritura. La clase `ConexionTxt` podrá contener atributos y/o métodos adicionales, en caso de ser necesario.

6. Creación de las interfaces `IDAOAlumnos` e `IDAORegistros`, que se usarán en el contexto del patrón ***Abstract Factory*** para permitir migrar de mecanismo de persistencia. Los cabezales de sus métodos serán los mismos que ya poseen las clases DAO hasta el momento.

7. Adaptación de las clases `DAOAlumnos` y `DAORegistros` de modo que ahora implementen las interfaces anteriores. Sus métodos ya implementados no deberían sufrir cambios. Además, en la clase `Alumno`, cambiar el atributo `DAORegistros` por uno de tipo `IDAORegistros`.

8. Implementación de las clases `DAOAlumnosTxt` y `DAORegistrosTxt`, las cuales también implementarán las interfaces `IDAOAlumnos` e `IDAORegistros` y definirán el acceso a la estructura de archivos usada como segundo medio de persistencia. Tendrán los mismos cabezales, pero en vez de persistir sobre la BD, lo harán sobre la estructura de archivos. Se permite definir clases y/o métodos auxiliares para leer y escribir en los archivos, siempre que se utilicen **únicamente** en forma interna a las clases `DAOAlumnosTxt` y `DAORegistrosTxt`.

9. Creación de la interface `FabricaAbstracta` e implementación de las clases `FabricaMySQL` y `FabricaTxt` correspondientes al patrón ***Abstract Factory*** presentado en el teórico. La `FabricaAbstracta` será utilizada del modo visto en clase y las clases `FabricaMySQL` y `FabricaTxt` instanciarán las clases DAO que acceden al medio concreto de persistencia que corresponde a cada una, también del modo visto en el curso. La fábrica se utilizará desde la `Fachada` para instanciar el DAO de Alumnos que corresponda y desde el método constructor de la clase `Alumno` para instanciar el DAO de Registros que corresponda.

Este material es de uso exclusivo para los cursos impartidos por Universidad de la Empresa